

hw1

February 8, 2026

```
[29]: # Part 1
@show W = [randn(16, 2), randn(16, 16), randn(1, 16)]
@show b = [zeros(16), zeros(16), zeros(1)]
NN(x, y) = W[3]*tanh.(W[2]*tanh.(W[1]*[x, y]+b[1])+b[2])+b[3]
```

```
W = [randn(16, 2), randn(16, 16), randn(1, 16)] = [[1.072463523471029
-1.93337643029648; 0.523231782287188 -0.5687099130889836; 0.8723099464588924
0.5440887971958834; -1.5788174180435557 1.0358593756981898; 1.0745309031591428
2.716474710593413; -0.11173506067553898 3.157960470204406; -0.6223919722963
-0.035788576223018106; 1.2786632389466093 -0.18007082850761055;
0.8643627187362519 0.584608102371713; 2.1343279273399984 0.35645715065885464;
0.18195731955191585 0.28663771713927333; 2.8956329115159614 -0.7233832211444994;
-0.3935329003666162 0.5130959866884243; -0.39425371059114805
-0.5897495157136425; -0.4359592274648933 -1.4773945423377242;
-1.1787290836494697 0.3639838286322558], [0.22857304263430372
-1.3639802457748034 0.4188815235255033 -1.7352807567270456 -0.7502394213822454
-1.5705313716723939 0.06258067244707509 0.3033975438601528 1.868815337363025
-0.9225377963124134 0.10651544574094976 0.8465173680631074 -2.083654449077786
-1.4210552333134128 1.1974680166857306 -0.774091841229659; -0.6742800838961333
-0.6867846217255729 0.758340315374628 0.5435102107805676 -1.16474683895914
-0.0888746499146521 1.2336812187112312 -0.2505400813140175 1.1391816180482177
-0.23427324015862022 0.36439414664825864 -0.4747764302070185 -0.6853850842409052
-0.19779647119820234 -1.9502546823154776 2.211552155648479; -0.5497693201268078
0.4512966487424299 -0.2934583388904725 -0.01905760592043718 0.14349030753344927
1.709663665596852 -1.5603571363652937 -1.0461725188798405 -0.9596769082188868
-0.43203579121000335 0.062040967657990255 -1.6759505856099361
-0.6096898344028704 -0.22841302940270578 0.4152179371787738
0.009555875062809348; 0.14626864705108142 -0.7615236363031224 0.7540367823153918
-0.6114128178049888 1.9064621400177182 -0.7595366932541744 -0.326523558574027
1.0376584262345288 2.2373389180774916 0.27971396947038873 1.5004128946193498
0.061814308646693766 0.936870687353344 -0.31157031768426197 0.10089374814238389
-0.445624263604792; -0.7322968771576719 -0.9970046398070026 0.5202661828257242
-0.20546884321451947 0.801619464902455 1.8977652180646722 0.5211792704212909
0.4758099785171682 0.8988384626710615 0.21009467833899084 -1.7173750149579738
0.3878744063343732 -1.2085688505642478 0.17630606848026262 1.6404479241617802
1.05216663494359; 0.9437072910628239 -1.0984783012571806 -1.8821359721635305
1.216898762553141 0.20870698441000768 -0.2720016977160388 -0.2429570707861418
-0.34573890411348474 -0.9115442124675794 0.05361519235738634
```

-0.14739233765624932 1.8463411879749776 -1.39989887174139 0.5638997669355885
 2.477941358305786 0.037414542081555605; -0.9264341526297409 -1.8181054145394877
 0.46992117530474803 -1.1288140197772931 0.0879081633691953 -1.0306928063141514
 -0.44441958619451843 -1.4150143148839889 1.5682545360846447 1.3730346242413112
 0.642577517198462 0.11943267515057342 -0.8082719261830034 -0.7506080897474968
 -0.5110233459643935 -0.3789140155153883; 1.2243370751998786 -0.3662454065231089
 0.5261210173755005 -0.053104433539232124 -1.0763134940735721 -0.3675368575584295
 -0.31704265681468835 1.4733753664917484 1.0086996297514075 0.2975433439225892
 -0.5551426603569594 0.20706819237012267 -1.3789436149786052 -0.376753652875111
 0.19023914127411737 -0.02051700980447247; -0.1242511117670282 0.8708905601658483
 0.8569564107936698 0.23927016929056133 0.17623432762557867 -2.78856932945406
 -1.4517661264404773 -0.7669387801634608 0.2950249528778902 -0.23182167379091886
 1.0487297831879636 1.0410417269916634 0.8757948579996819 0.4967606954576167
 -0.9348193415237523 0.3209480404450402; 0.5995858225879523 -0.6168939147684726
 -1.1353224211993573 0.0838396186831219 0.9889679383214558 -1.2437426497276034
 0.3475885388751365 0.7082214951856604 -0.34322826158966274 0.965359163975398
 0.048623457685156524 0.8413260703331398 -0.12731588253630546 0.8398638290130253
 0.9086159831618943 -0.24377826251659535; -0.06592461919423773 0.1741366198443939
 0.12967905453039444 -1.0595837531165648 -0.13881240501021083 -1.4410611746414064
 -0.5698081083422134 1.3731862504079047 1.168824736098621 -0.8579623457025002
 1.6129908596055755 1.0503746167515229 1.5705622570145201 -1.476177359896388
 0.8215135758882474 0.9555589841853211; -0.8058153622857578 -0.5107278634678212
 -1.0313728592550473 0.09475653031702368 0.19626519977254234 -0.5246434699537026
 0.5761855537639362 0.1544646558071401 -0.1823646283661945 0.2751437748014475
 0.579824237155091 1.160859217831959 -0.24164486493088788 1.4482802684452831
 -0.955441277445935 0.14458294758068005; -1.902405349119008 0.48643911577869764
 -1.1563892621384022 1.1963498869634186 -0.912920523064001 0.3012084105997926
 -1.2663595489838124 -0.5162454957969019 -1.4371447814038087 -0.6631368394618589
 0.08104841727364413 1.1407304039670125 -1.9625385580488237 -0.8731427710555713
 -0.40652240284055396 0.2968639294017946; -1.1271653910624357 -0.3881101173161601
 -0.8951672205403831 -0.7417692404902865 0.5566781309141883 -0.09917761559838836
 2.094657455027223 -0.25754614223558603 -0.2995465849253439 -1.2177450920014792
 1.025368309365438 -1.4850708643190056 -0.4748780165307548 0.921905086780302
 0.13063930136409868 0.31333454973604574; -0.17698081945580452
 -0.23481124806241782 0.02424529533646108 0.7183385722487655 1.9489085505148263
 -0.3372562440893441 -0.037075789922335976 1.146656419341395 0.5889771940123385
 -1.6127156082305791 -0.362006668514114 -0.46064898609918076 2.0170797478739266
 1.3303791454475116 -1.313345370089323 0.08044719551543153; 0.4909240166388423
 0.6540452298300519 0.16058134266001395 -1.8249365577334682 -0.468290653798421
 0.8609269335797216 -1.7915939985606943 -0.007768011822688123 0.3579722970808367
 1.601103877282016 -0.045924556795487355 -1.663785105594727 -0.12562558751007466
 -0.059921445415502476 0.6751999924146993 -0.04753238302091643],
 [-1.4282197035781048 -0.23423934935405327 -0.3173555772408637
 -0.2958149376870086 3.0488865040760023 0.8815446450307287 -5.683867613675353e-5
 -1.8143725197258067 0.8984789872426421 -0.5717398738845426 -1.4156311276427103
 -1.0496001590243387 1.3795501315268581 0.9775607015242908 0.2748936759723145
 -0.2900605325801879]]

b = [zeros(16), zeros(16), zeros(1)] = [[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,

```
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0], [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0], [0.0]]
```

NN (generic function with 1 method)

```
[30]: # Part 2
function NN_derivative(x, y, dx, dy)
    z1 = W[1] * [x, y] + b[1]
    z2 = W[2] * tanh.(z1) + b[2]
    W[3] * (sech.(z2) .^ 2 .* (W[2] * (sech.(z1) .^ 2 .* (W[1] * [dx, dy]))))
end
```

NN_derivative (generic function with 1 method)

```
[31]: # Part 3
function gradient_descent(f, df, u0, alpha; tol = 1e-10, maxiters = 1000,
    verbose = false)
    un = u0
    step = df(un) * alpha
    epoch = 0
    while sqrt(sum(abs2, step)) >= tol && epoch < maxiters
        epoch += 1
        if verbose
            println("Epoch: ", epoch, " Loss: ", f(un), " Step size: ",
                sqrt(sum(abs2, step)))
        end
        un = un - step
        step = df(un) * alpha
    end
    [un, f(un)]
end

f(x) = sum(abs2, x)
df(x) = 2 .* x

gradient_descent(f, df, [1.0, 1.0], 0.01)
```

2-element Vector{Any}:

```
[3.482867618819059e-9, 3.482867618819059e-9]
2.4260733700436688e-17
```

```
[32]: # Part 4: function definitions
using LinearAlgebra

nn_val(x, y) = first(NN(x, y))
nn_deriv_val(x, y, dx, dy) = first(NN_derivative(x, y, dx, dy))

const _h = 1e-6
nn_x(x, y) = nn_deriv_val(x, y, 1.0, 0.0)
```

```

nn_y(x, y) = nn_deriv_val(x, y, 0.0, 1.0)
nn_xx(x, y) = (nn_x(x + _h, y) - nn_x(x - _h, y)) / (2 * _h)
nn_yy(x, y) = (nn_y(x, y + _h) - nn_y(x, y - _h)) / (2 * _h)

g(x, y) = x * (1 - x) * y * (1 - y)
g_x(x, y) = y * (1 - y) * (1 - 2x)
g_xx(x, y) = -2 * y * (1 - y)
g_y(x, y) = x * (1 - x) * (1 - 2y)
g_yy(x, y) = -2 * x * (1 - x)

function u_pred(x, y)
    n = nn_val(x, y)
    n_x = nn_x(x, y)
    n_y = nn_y(x, y)
    n_xx = nn_xx(x, y)
    n_yy = nn_yy(x, y)
    g_ = g(x, y)
    gx_ = g_x(x, y)
    gy_ = g_y(x, y)
    gxx_ = g_xx(x, y)
    gyy_ = g_yy(x, y)
    u_xx = gxx_ * n + 2 * gx_ * n_x + g_ * n_xx
    u_yy = gyy_ * n + 2 * gy_ * n_y + g_ * n_yy
    (; u_xx, u_yy)
end

const n_colloc = 15
const _xs = [(i / (n_colloc + 1)) for i = 1:n_colloc]
const _ys = [(j / (n_colloc + 1)) for j = 1:n_colloc]

function poisson_loss_at(u_params)
    unpack!(u_params)
    residual_sq = 0.0
    for x in _xs
        for y in _ys
            (; u_xx, u_yy) = u_pred(x, y)
            rhs = sin(pi * x) * sin(pi * y)
            residual = u_xx + u_yy + rhs
            residual_sq += residual^2
        end
    end
    residual_sq / (n_colloc * n_colloc)
end

function pack!(W, b)
    vcat(vec(W[1]), vec(W[2]), vec(W[3]), vec(b[1]), vec(b[2]), vec(b[3]))
end

```

```

function unpack!(v)
    i = 0
    W[1][:] = v[(i+1):(i+32)];
    i += 32
    W[2][:] = v[(i+1):(i+256)];
    i += 256
    W[3][:] = v[(i+1):(i+16)];
    i += 16
    b[1][:] = v[(i+1):(i+16)];
    i += 16
    b[2][:] = v[(i+1):(i+16)];
    i += 16
    b[3][:] = v[(i+1):(i+1)];
    i += 1
    nothing
end

const _eps = 1e-7
function poisson_loss_grad(u_params)
    grad = similar(u_params)
    f0 = poisson_loss_at(u_params)
    for i in eachindex(u_params)
        u_plus = copy(u_params);
        u_plus[i] += _eps
        u_minus = copy(u_params);
        u_minus[i] -= _eps
        grad[i] = (poisson_loss_at(u_plus) - poisson_loss_at(u_minus)) / (2 * _eps)
    end
    grad
end

```

poisson_loss_grad (generic function with 1 method)

```

[ ]: # Part 4: training
u0 = pack!(W, b)

alpha = 1e-2
result = gradient_descent(poisson_loss_at, poisson_loss_grad, u0, alpha; tol = 1e-4, maxiters = 500, verbose = true)
u_opt, loss_opt = result
unpack!(u_opt)

using Serialization

@show my_data = Dict{:W => W, :b => b)
serialize("hw1.jls", my_data)

```

```
my_data_from_serialized = deserialize("hw1.jls")
@assert my_data == my_data_from_serialized
```

```
[33]: # Part 4: loading parameters
using Serialization

my_data_from_serialized = deserialize("hw1.jls")
W = my_data_from_serialized[:W]
b = my_data_from_serialized[:b]
u_opt = pack!(W, b)

loss_opt = poisson_loss_at(u_opt)
println("Final loss: ", loss_opt)
```

Final loss: 0.007574933663770549

```
[34]: # Visualization
u_exact(x, y) = sin(pi * x) * sin(pi * y) / (pi^2) / 2

using Plots

n_plot = 40
x_plot = range(0, 1; length = n_plot)
y_plot = range(0, 1; length = n_plot)
Z_approx = [u_approx(x, y) for x in x_plot, y in y_plot]
Z_exact = [u_exact(x, y) for x in x_plot, y in y_plot]

p1 = heatmap(
    x_plot,
    y_plot,
    Z_approx';
    xlabel = "x",
    ylabel = "y",
    title = "PINN solution u(x,y)",
    aspect_ratio = 1,
)
p2 = heatmap(
    x_plot,
    y_plot,
    Z_exact';
    xlabel = "x",
    ylabel = "y",
    title = "Exact solution u(x,y)",
    aspect_ratio = 1,
)
p3 = heatmap(
    x_plot,
    y_plot,
```

```

(Z_approx .- Z_exact)';
xlabel = "x",
ylabel = "y",
title = "Error (approx - exact)",
aspect_ratio = 1,
)

plot(p1, p2, p3; layout = (1, 3), size = (1600, 350))

```

